INFORME DE LAS PRÁCTICAS DE ALGORITMIA

Jaime González Díaz
UO306054
Algoritmia

ÍNDICE

PRACTICA P0 2
PRACTICA P11 10
Toma de tiempos de ejecución 10
Crecimiento del tamaño del problema 10
Toma de tiempos pequeños (<50ms) 10
PRACTICA P12 13
Ejercicio 1 13
Ejercicio 2 13
Ejercicio 3 14
Ejercicio 4 14
Ejercicio 5 15
PRACTICA P2 16
EJERCICIO 1: algoritmo burbuja 16
EJERCICIO 2: algoritmo selección 16
EJERCICIO 3: algoritmo inserción 17
EJERCICIO 4: algoritmo rapido 17
EJERCICIO 5: algoritmo rápidoBajos vs inserciónBajos 18
EJERCICIO OPCIONAL 19
PRACTICA P3 21
DyV por SUSTRACCIÓN 21
DyV por división: 23
Dos ejemplos básicos: 25
Practica DyV 26
Práctica P4 28
Ejercicio 1: 28
EJERCICIO 2 28
Práctica P5 29
Definición del estado: 29
Relación de recurrencia: 29
Complejidad: 30
Implementación: 30
Práctica P6 30
TIEMPOS DEL ALGORITMO DE LOS CONTENEDORES: 30
Práctica P6 EXTRA 31
TABLA DE TIEMPOS ALGORITMO DE LAS N REINAS 31
Práctica P7 32
Heurístico aplicado 32
Tabla de tiempos comparativa 32

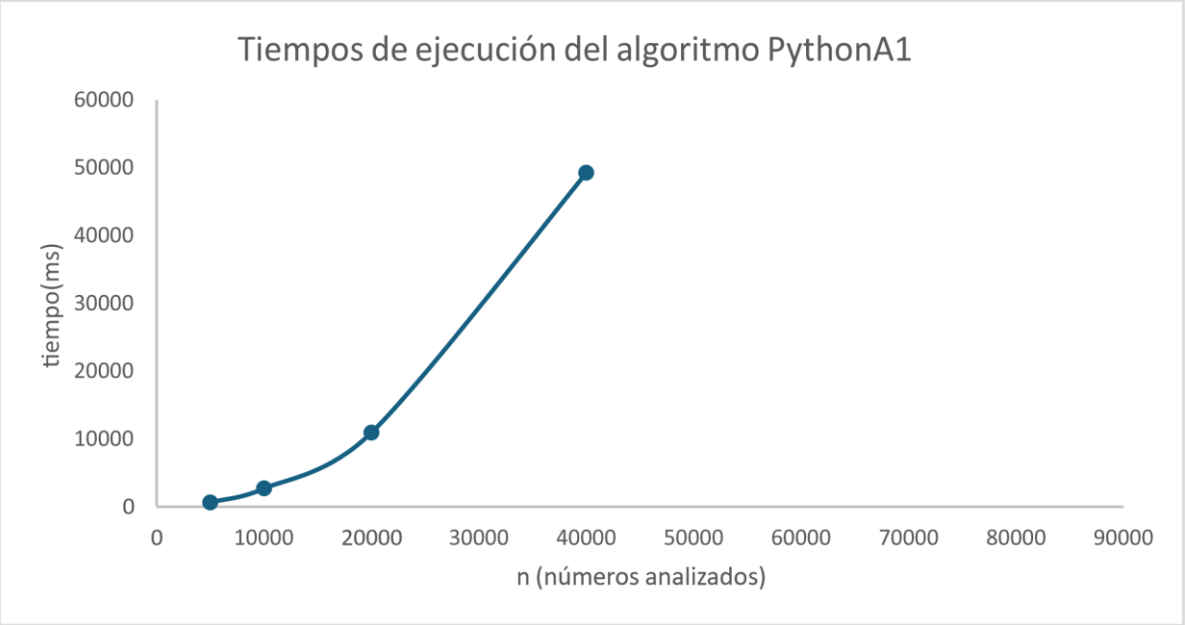
Jaime González Díaz
UO306054

PRACTICA P0

FACTOR1

n	t(ms)
5000	643
10000	2719
20000	10959
40000	49259
80000	

(FdT)

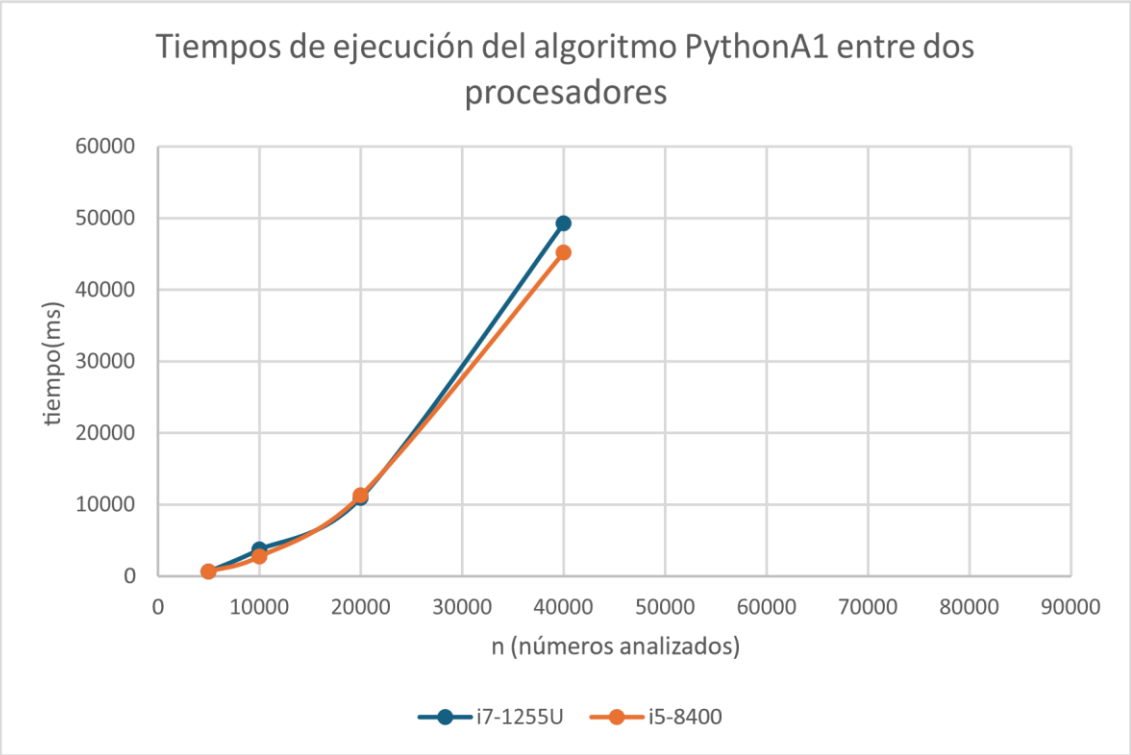


FACTOR2

Jaime González Díaz
UO306054

n	i7-1255U	i5-8400
5000	643	663
10000	3729	2755
20000	10959	11285
40000	49259	45201
80000		

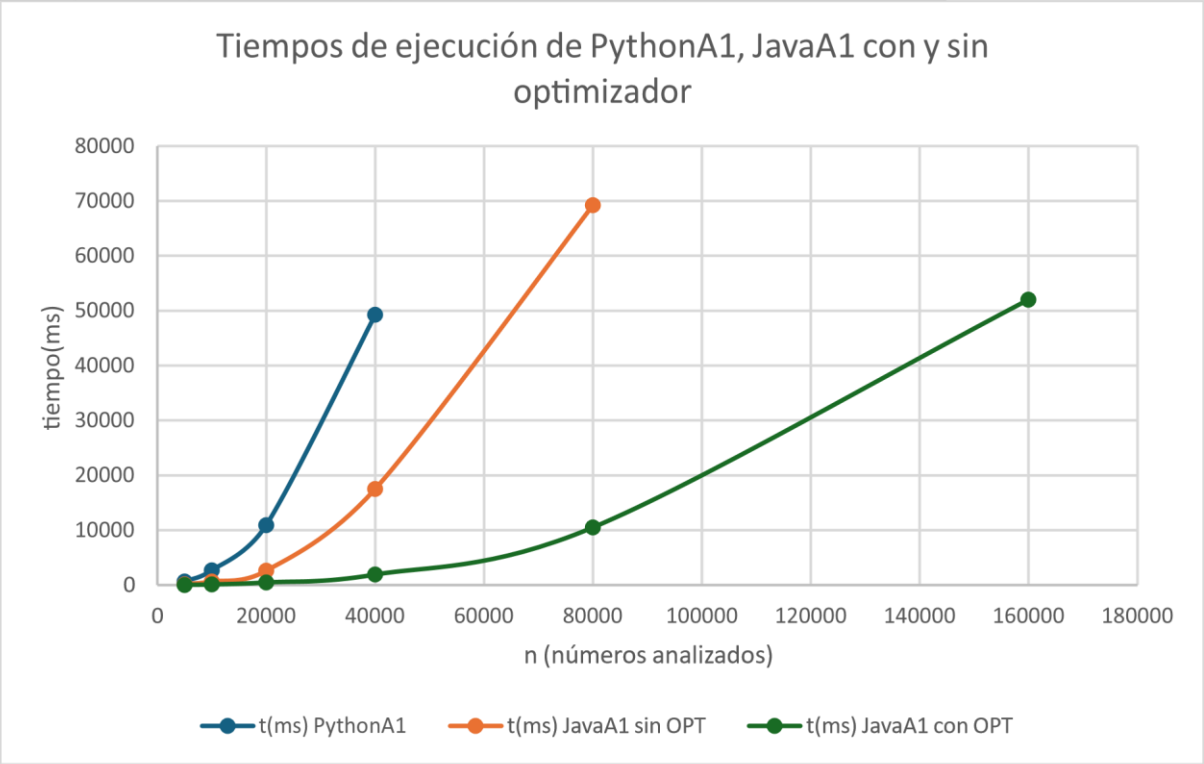
(FdT)



Jaime González Díaz
UO306054

FACTOR3

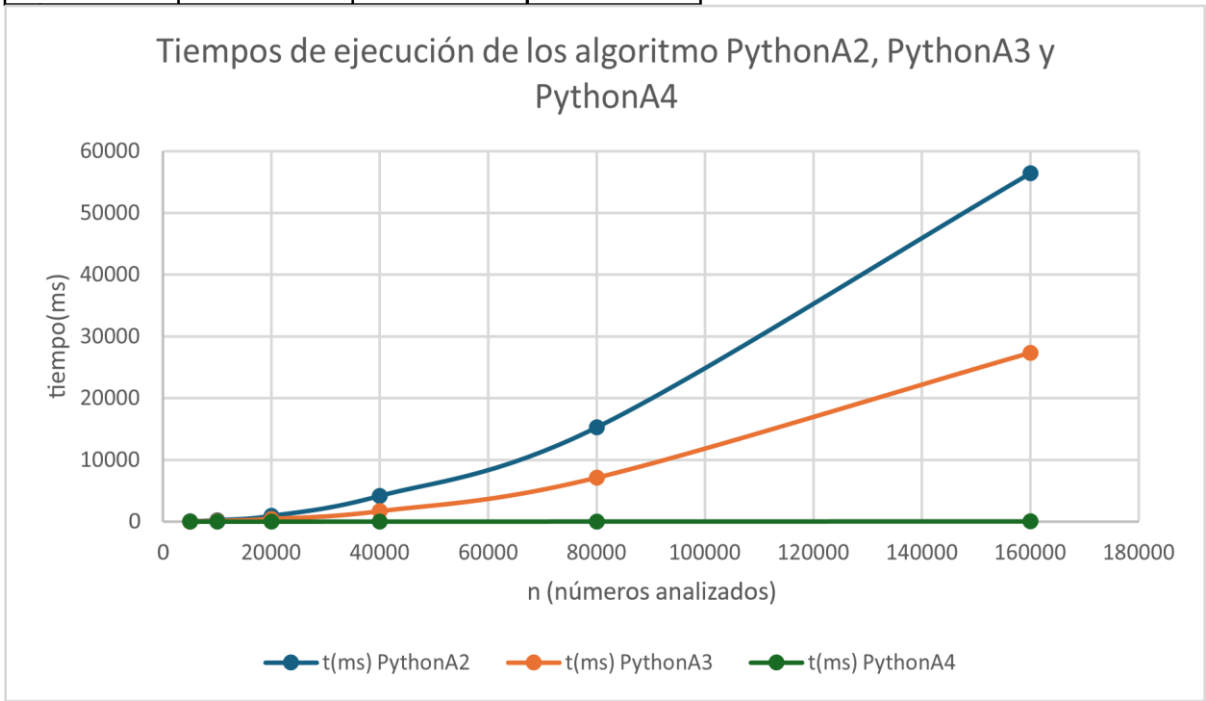
n	t(ms) PythonA1	t(ms) JavaA1 sin OPT	t(ms) JavaA1 con OPT
5000	643	169	47
10000	2719	655	120
20000	10959	2634	488
40000	49259	17528	1927
80000		69238	10522
160000			52037



FACTOR4

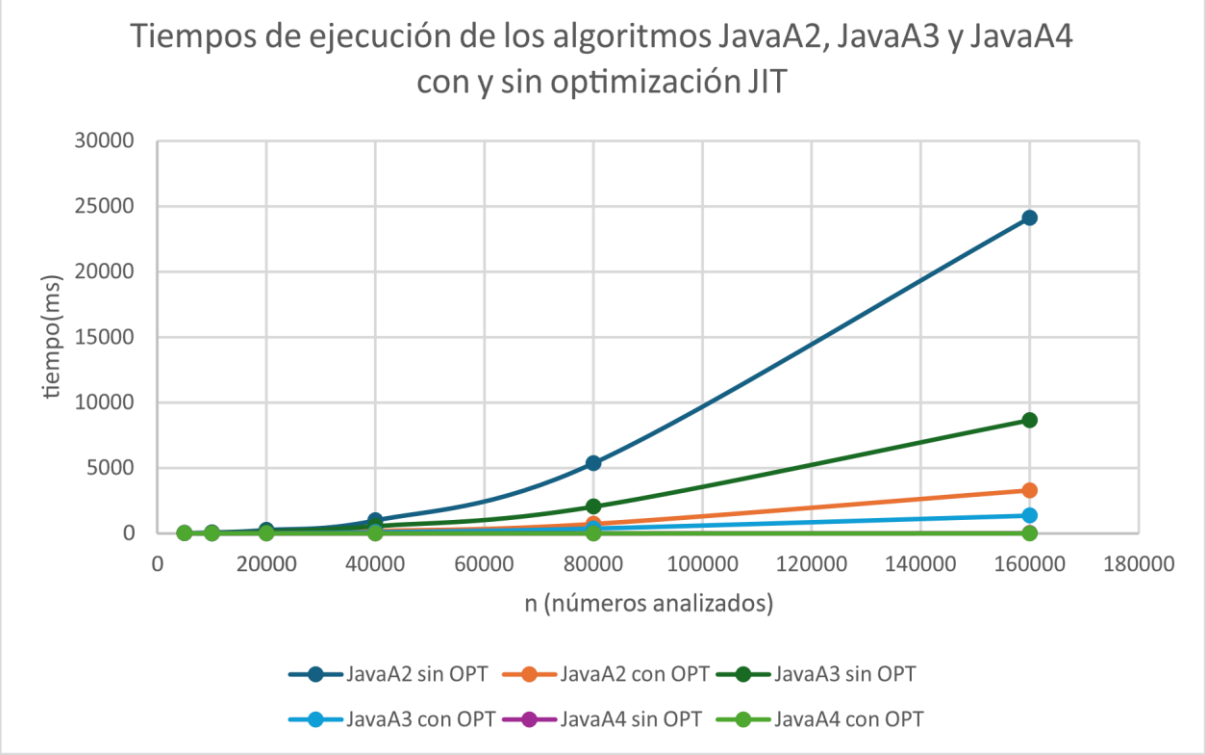
Jaime González Díaz
UO306054

Comparativa PythonA2, PythonA3, PythonA4			
n	t(ms) PythonA2	t(ms) PythonA3	t(ms) PythonA4
5000	71	29	1
10000	242	118	2
20000	938	451	4
40000	4181	1704	11
80000	15271	7125	24
160000	56426	27362	41



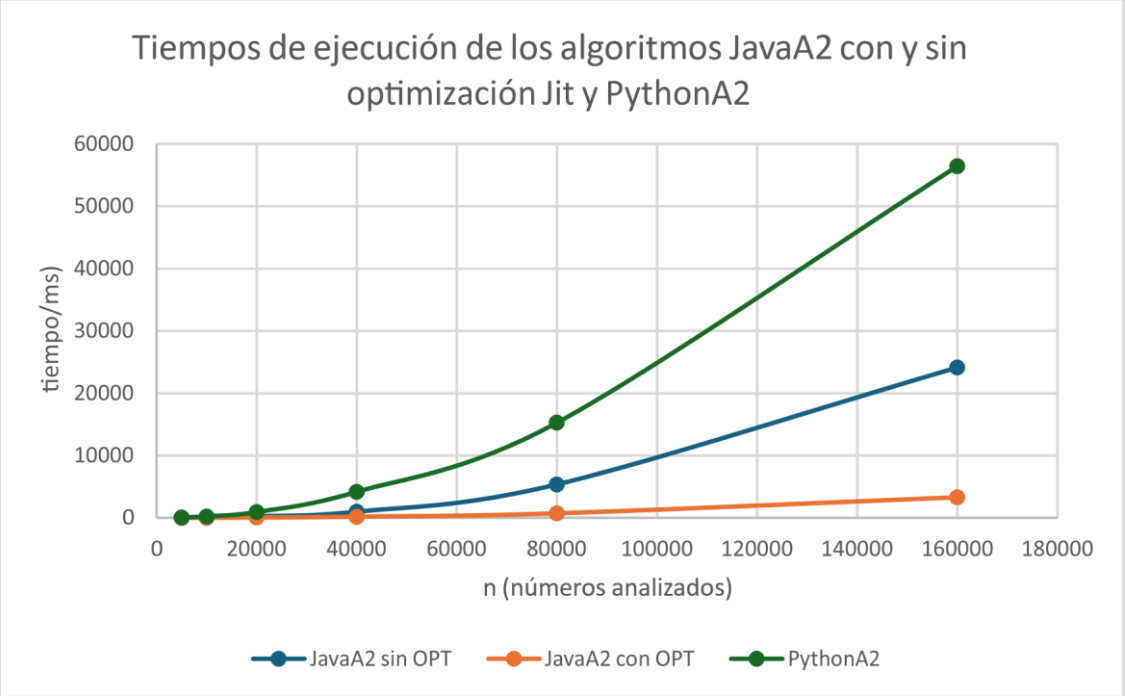
Jaime González Díaz
UO306054

Comparativa JavaA2, JavaA3 y JavaA4 con y sin optimización JIT						
n	JavaA2 sin OPT	JavaA2 con OPT	JavaA3 sin OPT	JavaA3 con OPT	JavaA4 sin OPT	JavaA4 con OPT
5000	35	26	29	21	17	16
10000	75	15	38	8	1	1
20000	258	53	148	26	1	1
40000	990	192	554	98	3	1
80000	5367	726	2044	376	5	3
160000	24128	3293	8654	1362	15	5



Jaime González Díaz
UO306054

Comparativa JavaA2 sin OPT, JavaA2 con OPT y PythonA2			
n	JavaA2 sin OPT	JavaA2 con OPT	PythonA2
5000	35	26	71
10000	75	15	242
20000	258	53	938
40000	990	192	4181
80000	5367	726	15271
160000	24128	3293	56426

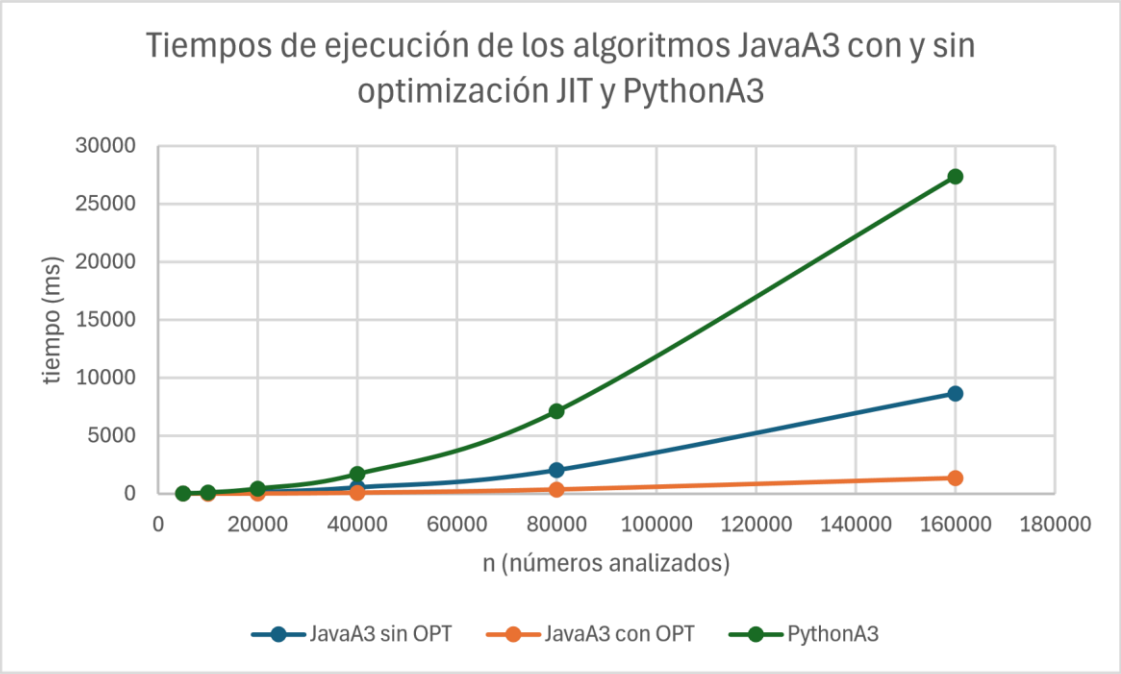


Comparativa JavaA3 sin OPT, JavaA3 con OPT y PythonA3			
n	JavaA3 sin OPT	JavaA3 con OPT	PythonA3
5000	29	21	29

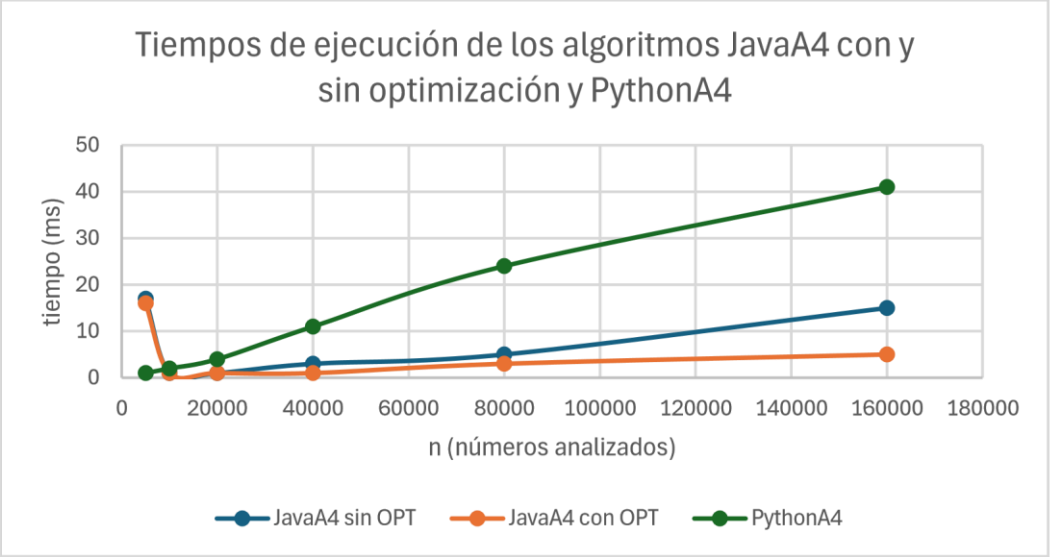
Jaime González Díaz
UO306054

10000	38	8	118
20000	148	26	451
40000	554	98	1704
80000	2044	376	7125
160000	8654	1362	27362

Jaime González Díaz
UO306054



Comparativa JavaA4 sin OPT, JavaA4 con OPT y PythonA4			
n	JavaA4 sin OPT	JavaA4 con OPT	PythonA4
5000	17	16	1
10000	1	1	2
20000	1	1	4
40000	3	1	11
80000	5	3	24
160000	15	5	41



Jaime González Díaz
UO306054

PRACTICA P11

Toma de tiempos de ejecución

Ejercicio 1: Esta forma de contar la podremos usar durante 5,8*10^8 – 56 años. Este valor lo conseguimos tras aplicar la formula t2=f(n2)/f(n1)*t1.

Ejercicio 2: el tiempo medido a veces sale 0 debido a factores como la velocidad del procesador que ejecuta el código en un tiempo menor a 1 milisegundo. También puede ser porque si los valores t2 y t1 pueden ser similares, al hacer la resta el resultado queda aproximadamente en 0.

Ejercicio 3: el tamaño n del problema donde los tiempos empiezan a ser fiables (mayores a 50ms) son aproximadamente 8 millones.

Crecimiento del tamaño del problema

Tiempos de ejecución de Vector 3

```
C:\Users\jaime\Desktop\alg_GonzalezDiazJaime306054\p11>java -Xint p11.Vector3
n   tTiempo
10000   0
20000   0
40000   1
80000   1
160000  1
320000  3
640000  7
1280000 13
2560000 23
5120000 56
10240000        104
20480000        187
40960000        375
81920000        804
```

Toma de tiempos pequeños (<50ms)

Ejercicio 1: tiempos del vector 4 con 1, 10, 1000

Jaime González Díaz
UO306054

```
C:\Users\jaime\Desktop\alg_GonzalezDiazJaime306054\p11>java -Xint Vector4.
java 1
repeticiones = 1
n        Tiempo
10000   0
20000   0
40000   0
80000   1
160000  1
320000  2
640000  6
1280000 8
2560000 18
5120000 36
10240000        71
20480000        137
40960000        286
81920000        578

Fin de la medicion de tiempos *****
```

```
C:\Users\jaime\Desktop\alg_GonzalezDiazJaime306054\p11>java -Xint Vector4.
java 10
repeticiones = 10
n        Tiempo
10000   0
20000   2
40000   4
80000   6
160000  13
320000  23
640000  42
1280000 85
2560000 174
5120000 356
10240000        705
20480000        1401
40960000        2826
81920000        5655

Fin de la medicion de tiempos *****
```

```
C:\Users\jaime\Desktop\alg_GonzalezDiazJaime306054\p11>java -Xint Vector4.
java 100
repeticiones = 100
n        Tiempo
10000   7
20000   13
40000   29
80000   55
160000  112
320000  222
640000  438
1280000 880
2560000 1753
5120000 3534
10240000        7025
20480000        14098
40960000        28073
81920000        56841

Fin de la medicion de tiempos *****
```

Ejercicio 1: Si el tamaño del problema se duplica por dos, el tiempo de ejecución también se duplica.

Ejercicio 2: Si el tamaño del problema se multiplica por k, entonces el tiempo de ejecución también aumenta proporcionalmente a k.

Ejercicio 3: Sí, los tiempos obtenidos (reflejados en Tsuma) son esperados con la complejidad lineal O(n).

Estos tiempos se han medido en un ordenador con procesador Intel i7-12700H y con una memoria RAM de 16GB.

Jaime González Díaz
UO306054

n	Tsuma	Tmaximo
10000	67	86
20000	132	176
40000	266	362
80000	551	736
160000	1072	1468
320000	2212	2826
640000	4403	5644
1280000	8731	11275
2560000	17578	22538
5120000	35255	57359
10240000	FdT	FdT
20480000	FdT	FdT
40960000	FdT	FdT
81920000	FdT	FdT

n	Tcoincidencias1	Tcoincidencias2
10000	877	124
20000	3475	202
40000	13834	416
80000	55481	807
160000	FdT	2180
320000	FdT	6264
640000	FdT	9735
1280000	FdT	17637
2560000	FdT	38641
5120000	FdT	FdT
10240000	FdT	FdT
20480000	FdT	FdT
40960000	FdT	FdT
81920000	FdT	FdT

Los tiempos sí encajan con lo esperado de la complejidad de cada código.

Jaime González Díaz
UO306054

PRACTICA P12

Ejercicio 1

(Usando 1 repetición)

Pondré FdT si pasa de 2 minutos

n	TBucle1	TBucle2	TBucle3	TBucle4
100	0	0	1	2
200	0	1	6	9
400	0	7	29	77
800	0	24	130	670
1600	0	95	525	7716
3200	1	452	2217	FdT
6400	2	1930	16622	FdT
12800	3	17587	85165	FdT
25600	11	67206	FdT	FdT
51200	14	FdT	FdT	FdT

Los tiempos tienen sentido de acuerdo a la complejidad temporal de cada uno, siendo O(nlogn), O(n^2logn), O(n^2logn) y O(n^3), respectivamente.

Ejercicio 2

(Ya están implementados bucle5 6 y 7 pero comprobar si están bien)

Pondré FdT cuando pase de 3 minutos (En bucle7 puse una más porque me parecía muy poco solo dos iteraciones)

n	TBucle5	TBucle6	TBucle7
100	9	119	935
200	50	1076	25325
400	221	14896	309816
800	1048	1037	FdT
1600	7750	136939	FdT

Jaime González Díaz
UO306054

3200	40724	FdT	FdT
6400	123965	FdT	FdT

Los tiempos concuerdan con lo esperado ya que las mediciones siguen la misma tendencia de la complejidad de sus respectivas funciones.

Ejercicio 3

n	TBucle1 (t1)	TBucle2 (t2)	T1/T2
100	0	0	0
200	0	1	0
400	0	7	0
800	0	24	0
1600	0	95	0
3200	1	452	0,002
6400	2	1930	0,001
12800	3	17587	1,7*10^-4
25600	11	67206	1,7*10^-4
51200	14	FdT	FdT

Los tiempos tienen sentido ya que el bucle 1 es de una complejidad menor que el bucle 2.

Ejercicio 4

Asumimos t=0 como t=1 para dividir

n	TBucle3	TBucle2	T3/T2
100	1	0	1
200	6	1	6

400	29	7	4,14
800	130	24	5,41
1600	525	95	5,52
3200	2217	452	4,904
6400	16622	1930	8,61
12800	85165	17587	4,85
25600	FdT	67206	FdT
51200	FdT	FdT	FdT

El tiempo y su cociente concuerdan ya que aunque sean algoritmos de la misma complejidad, el bucle3 tardará más debido a la implementación del código, que matemáticamente se puede comprobar que va a dar resultados mayores que el bucle2.

Ejercicio 5

(Uso 10 repeticiones para Python y 1 para Java)

Marcaremos FdT aquellos tiempos que superen los 5 minutos.

Asumimos t=0 como t=1 para la división

n	Bucle4 py (t1)	Bucle4 java sin OPT (t2)	t1/t2
100	12	0	12
200	82	12	6,833
400	675	92	7,33
800	8925	630	14,16
1600	114080	7319	15,58
3200	FdT	72378	FdT

6400	FdT	FdT	FdT

Los tiempos tienen sentido ya que Python es interpretado, lo que hace que tarde más en ejecutarse, eso sumado a la complejidad cúbica del código, hará que el bucle 4 de Python tarde mucho más que el código de Java.
Jaime González Díaz UO306054

PRACTICA P2

EJERCICIO 1: algoritmo burbuja

n	T ordenado	T invertido	T aleatorio
10000	782	1579	1575
20000	3101	6406	6376
40000	12580	43871	33782
80000	50497	175189	177151
160000	343380	FdT	FdT
320000	FdT	FdT	FdT
640000	FdT	FdT	FdT
1280000	FdT	FdT	FdT

Los tiempos tienen sentido ya que el algoritmo burbuja tiene complejidad cuadrática en todos los casos. Sin embargo es más rápido en el caso de que esté ordenado porque no realiza ningún cambio entre pares de elementos y eso en listas con tamaños n muy grandes supone una diferencia de tiempo considerable, ya que solo recorre un vector ordenado en dos iteraciones anidadas.

EJERCICIO 2: algoritmo selección

n	T ordenado	T invertido	T aleatorio
10000	730	736	746
20000	3505	3547	3560
40000	20175	20970	19941
80000	80587	80537	80142
160000	FdT	FdT	FdT
320000	FdT	FdT	FdT
640000	FdT	FdT	FdT
1280000	FdT	FdT	FdT

Los tiempos tienen sentido ya que el algoritmo siempre es en complejidad O(n^2) sea el caso que sea. Por eso todos tienen tiempos tan similares y de orden cuadrático.

EJERCICIO 3: algoritmo inserción

n	T ordenado	T invertido	T aleatorio
10000	1	631	626
20000	1	2569	2522
40000	1	16977	10106
80000	2	67970	40655
160000	3	277248	259251
320000	5	FdT	FdT
640000	11	FdT	FdT
1280000	21	FdT	FdT

Los resultados tienen sentido ya que el algoritmo inserción se caracteriza por tener una complejidad O(n^2) menos en el mejor caso donde la lista está ordenada, donde no entra en la segunda iteración anidada y se convierte en un algoritmo de complejidad lineal, dando por tanto resultados bajos de complejidad O(n).

EJERCICIO 4: algoritmo rapido

n	T ordenado	T invertido	T aleatorio
10000	5	8	7
20000	10	16	15
40000	21	32	29
80000	43	67	68
160000	87	132	138
320000	217	309	303
640000	403	664	670
1280000	914	1436	1454

1) Los tiempos obtenidos tienen sentido ya que el Quicksort se caracteriza en que la complejidad se mantiene O(nlogn) en todos los casos posibles gracias a su uso de las llamadas a recursividad, hace menos comparaciones si la lista está ordenada y por eso en dicho caso sí tarda menos.
2) Calculando con la formula T(n) de cada algoritmo:
    a. Burbuja: t2=((n2)^2/(n1)^2)*t1. Usando n1=80.000 y t1=177151ms, calculamos que t2=185.756.287.976ms=**215,64 días.**

    b. Selección: t2=((n2)^2/(n1)^2)*t1. Usando n1=80000 y t1=80142ms, calculamos que t2=84.035.000.792ms=**97,55 días.**

Jaime González Díaz
UO306054

c. Inserción: t2=((n2)^2/(n1)^2)*t1. Usando n1=160000 y t1=259251ms, calculamos que t2=67.961.103.144ms=**78,89 días**.

d. Rápido: t2=(log(n2)/log(n1))*t1. Usando n1=1.280.000 y t1=1454ms, haciendo log(81920000) calculamos que T(n2)=120.573ms=**0,00014 días.**

EJERCICIO 5: algoritmo rápidoBajos vs inserciónBajos

(Se han usado 100000 repeticiones)

n	T rapido	T insercion	Ttrapido/Tinsercion
10	218	128	1,703125
15	345	207	1,66666667
20	503	347	1,44956772
25	676	451	1,49889135
30	837	699	1,19742489
35	1019	919	1,10881393
40	1241	1209	1,02646816
45	1435	1510	0,95033113
50	1565	1774	0,88218715
55	1791	2125	0,84282353
60	1993	2534	0,78650355
65	2141	2947	0,72650153
70	2441	3407	0,7164661
75	2606	3850	0,67688312

80	2837	4407	0,64374858
85	3029	4845	0,6251806
90	3245	5593	0,58018952
95	3444	6109	0,56375839
100	3591	6712	0,53501192

Los tiempos tienen sentido ya que el Quicksort realiza llamadas recursivas que en tamaños muy pequeños ralentizan el proceso, mientras que el Inserción realiza soluciones directas de forma iterativa. Sin embargo, a partir de un tamaño n (40) la llamada recursiva es más eficiente ya que el divide y vencerás en estos tamaños es más práctico a la hora de solucionar problemas para n más pequeños que una solución iterativa cuadrática

EJERCICIO OPCIONAL

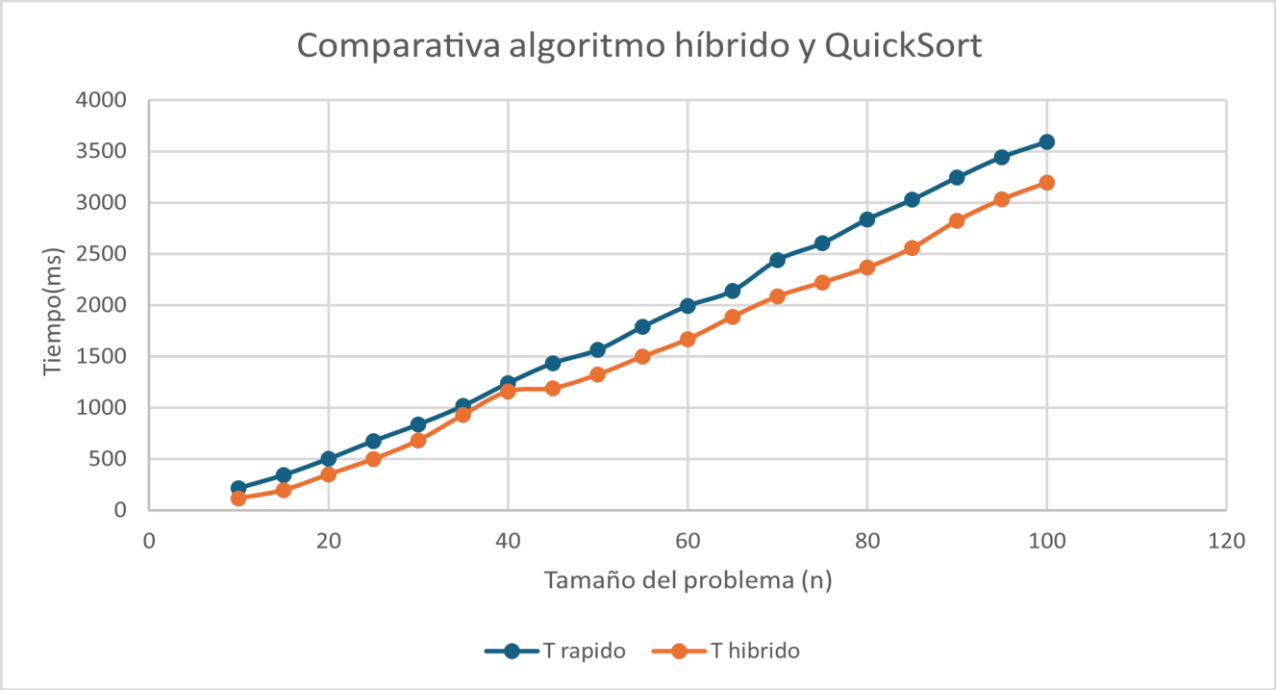
n	T rapido	T hibrido	Ttrapido/Thibrido
10	218	118	1,847458
15	345	199	1,733668
20	503	351	1,433048
25	676	502	1,346614
30	837	682	1,227273
35	1019	933	1,092176

Jaime González Díaz
UO306054

40	1241	1160	1,069828
45	1435	1192	1,203859
50	1565	1325	1,181132
55	1791	1501	1,193205
60	1993	1668	1,194844
65	2141	1888	1,134004
70	2441	2087	1,169621
75	2606	2223	1,17229
80	2837	2367	1,198564
85	3029	2560	1,183203
90	3245	2824	1,149079
95	3444	3032	1,135884
100	3591	3195	1,123944

Los tiempos Thibrido representan el código contenido en el archivo AlgoritmoOpcional.java, donde tras ver que el algoritmo de inserción es más rápido que el rápido hasta aproximadamente n=40, apliqué un algoritmo híbrido que utiliza durante los primeros 40 tamaños el algoritmo de inserción y a partir de ahí utiliza el rápido, implementando toda la lógica necesaria de ambos algoritmos en la misma clase, dando así tiempos más rápidos para todos los n desde 10 hasta 100, donde se acercan más los tiempos entre ambos algoritmos cuanto más aumenta el tamaño del problema al estar aplicando el mismo algoritmo de ordenación.

Incluso en los tiempos más altos mi algoritmo híbrido sigue siendo mas eficiente ya que durante las llamadas recursivas, cuando llega a una partición de tamaño inferior al cruce (en este caso, 40), deja de emplear Quicksort y emplea algoritmo de inserción por lo antes dicho, que es más rápido para estos tamaños.



(Sé que me dijiste que llamase al Rapido e Insercion desde sus clases pero me empeñé en querer rascar tiempos más pequeños)

PRACTICA P3

DyV por SUSTRACCIÓN:

1. Las complejidades de Sustraccion1, 2 y 3 son: O(n), O(n^2) y O(2^n). Lo cual podemos averiguar tras aplicar el teorema maestro sobre cada algoritmo. Los tiempos sí tienen sentido en cada complejidad ya que para cada secuencia de n el tiempo aumenta correctamente en función de la complejidad.
2. En Sustraccion1 con parámetro 1 la clase aborta para n=16000 y para Sustraccion2 con parámetro 1 también aborta para n=16000. Esto es debido a que la pila de llamadas de funciones de Java desborda tras una cantidad excesiva de llamadas recursivas que se acumulan a partir de dicho n y, al no llegar al caso base antes de llenar la pila, esta desborda y devuelve un StackOverflowError.

3. En Sustraccion3, sabiendo que el tiempo de ejecución de n=20 es de 61ms, sabiendo además que la complejidad del algoritmo es de O(2^n). Para n=80 aplicamos el cálculo: t2=(f(n2)/f(n1))*t1= ((2^80)/(2^20))*61 = 6,1*10^19ms, es decir, unos 2.228.530.407 años.

4.
```
public static void rec4 (int n)
{
    if (n<=0)
        cont++;
    else
    {
        for (int i=0;i<n;i++)
            cont++;
        for (int i=0;i<n;i++)
                for (int j=0;j<n;j++)
                    cont++ ;
        rec4 (n-1);

    }
}
```

n	t(ms)
100	3
200	25
400	194
800	1745
1600	12907
3200	99289

Como se puede ver, los tiempos de ejecución siguen una tendencia cúbica.

5.
```
public static void rec5 (int n)
{
    if (n<=0)
        cont++;
    else
    {
        rec5 (n-2);
        rec5 (n-2);
        rec5 (n-2);
    }
}
```

n	**Tiempo (ms)**
20	2
21	6
22	6

23	18
24	23
25	59
26	57
27	182
28	180
29	539
30	533
31	1610
32	1619
33	4817
34	4800
35	14358
36	14379
37	42893

Como se puede ver, los tiempos de ejecución siguen una tendencia que corresponde a la complejidad O(3^n/2)

6. En Sustraccion5, sabiendo que el tiempo de ejecución de n=36 es de 14.379ms, sabiendo además que la complejidad del algoritmo es de O(3^(n/2)). Para n=80 aplicamos el cálculo: t2=(f(n2)/f(n1))*t1= ((3^(80/2))/(3^(36/2)))*14.379 = (3^22)*14.379 = 4,51*10^14ms, es decir, unos 14.298 años.

DyV por división:

1. Las complejidades de Division1, Division2 y Division3 son O(n), O(nlogn) y O(n), respectivamente, lo cual podemos averiguar aplicando el teorema

maestro. Las complejidades coinciden con la tendencia de los tiempos en cada algoritmo.

2.
```
public class Division4
{

static long cont;

public static void rec4 (int n)
    {
        if (n<=0)
            cont++;
        else
        {
            for(int i=1; i<n;i++){
                for(int j=1;j<n;j++){
                    cont++;
                }
            }
            rec4 (n/2);
        }

    }
```

Podemos ver que la complejidad es de O(n^2) manteniendo la condición de a<b^k (1<2^2).

n	T(ms)
1000	14
2000	48
4000	192
8000	766
16000	3084
32000	12313
64000	49505

Como se puede ver, los tiempos siguen la tendencia correspondiente a la complejidad cuadrática.

3.
```
public class Division5
{

static long cont;

public static void rec5 (int n)
    {
        if (n<=0)
            cont++;
        else
        {
            for(int i=1; i<n;i++){
                cont++;
            }
            rec5 (n/2);
            rec5 (n/2);
            rec5 (n/2);
            rec5 (n/2);
        }

    }
```

Se cumple la complejidad cuadrática y la condición de a>b^k (4>2^1).

n	T(ms)
1000	41
2000	166
4000	653
8000	2613
16000	10482
32000	42147
64000	FdT

Como se puede ver, los tiempos coinciden con la complejidad cuadrática.

Dos ejemplos básicos:

1. Tiempos de Fibonacci (n=40) con parámetro 1:

n	Fib1()	Fib2()	Fib3()	Fib4()
T(ns)	333	714	826	5624*10^9

Tanto fib1, fib2 y fib3 son lineales O(n), pero sus tiempos son tan diferentes por la forma en la que están implementados a pesar de tener la misma complejidad. Fib1() es el más rápido porque solo usa tres variables locales

que van modificando su valor, operación que es más rápida que el resto de las operaciones empleadas por el resto de los métodos. Fib2() tarda más ya que emplea un array el cual se va escribiendo y leyendo en cada iteración del bucle, siendo una operación más lenta que cambiar el valor de una variable. Fib3() tarda más que las otras dos por el uso de múltiples llamadas recursivas, operación que tarda más que el resto al tener que gestionar la pila en todas las llamadas.

Fib4() tarda mucho más que el resto ya que no es lineal, su complejidad es exponencial O(1.6^n), lo que hace que para n=40, la diferencia de tiempos deje en claro que esta complejidad es mucho mas ineficiente que la lineal.

2. Tiempos de VectorSuma (n=100) con parámetro 1:

n	Sum1()	Sum2()	Sum3()
T(ns)	671	2279	4180

Tanto Sum1, Sum2 y Sum3 tienen complejidad lineal O(n). El método Sum1() realiza un bucle iterativo donde la operación es modificar el valor de una variable local, operación muy rápida. El método Sum2() emplea llamadas recursivas, aplicando Divide y Vencerás con sustracción, lo que alarga el tiempo de ejecución por su cantidad de llamadas y su gestión de pila en cada una de estas. Sum3() es el menos eficiente porque emplea llamadas recursivas aplicando Divide y Vencerás con división, que en este caso no es la mejor porque ira dividiendo el problema hasta llegar a tamaño 1, lo que hace que se llame casi al doble de veces a sí mismo en comparación al anterior, por eso su tiempo tan alto de ejecución.

Practica DyV

1. PRIMERA PARTE: PuntosTrivial.java (siendo el parámetro 100)

n	Tiempo(ms)
1024	81
2048	306
4096	1261
8192	4971
16384	27130
32768	108185

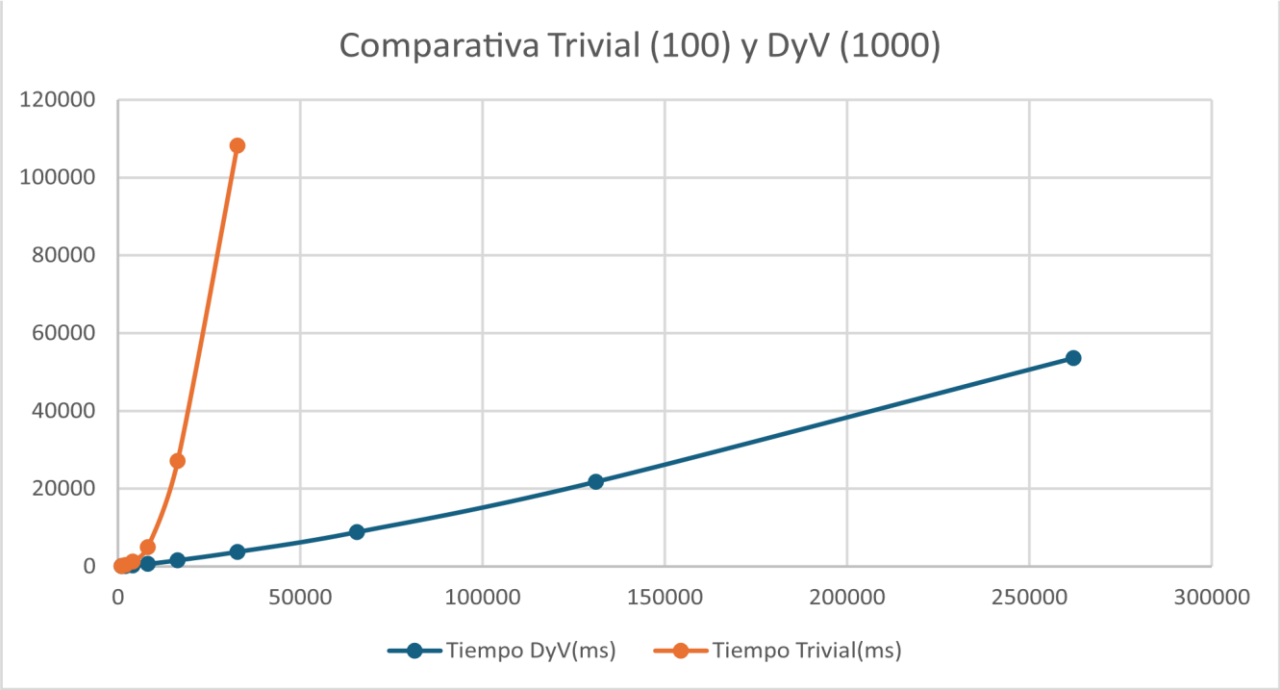
Los tiempos tienen sentido ya que la complejidad del algoritmo es cuadrática, igual a la tendencia de los tiempos medidos.

2. Segunda parte: PuntosDyV.java (siendo el parámetro 1000 ya que con 100 los tiempos se quedaban muy cortos).

n	Tiempo(ms)

1024	60
2048	104
4096	299
8192	641
16384	1586
32768	3752
65536	8834
131072	21777
262144	53586

Los tiempos tienen sentido ya que la complejidad del algoritmo es de O(nlogn), igual a la tendencia de los tiempos.
La versión Divide y Vencerás del algoritmo es muchísimo más eficiente que la versión trivial que era O(n^2). **Usando 900 repeticiones** más, los tiempos son muchos menores.



Jaime González Díaz
UO306054

Práctica P4

Ejercicio 1:

La implementación de Devorador.java tiene una complejidad lineal, O(n), ya que recorre todos los n nodos una única vez, a pesar de implementar más bucles dentro del bucle principal, estos no dependen de n ya que trabajan con variables independientes como el numero de vecinos que, independiente del tamaño n del grafo, puede ser como máximo 8. Lo mismo aplica para el bucle de colores donde el máximo no depende de n, depende de la cantidad de colores que tengamos en la lista externa de colores.
Los tiempos obtenidos para 1000 repeticiones son:

n	T(ms)
8	17
16	21
32	28
64	32
100	36
128	82
256	123
512	176
1024	414
2048	1006
4096	1983
8192	4525
16384	12140
32768	27330
65536	71237

Aunque la complejidad de los tiempos refleje una relación más bien de nlogn, como las estructuras de datos que usamos son Sets y Maps, que seguramente estén realizando funciones ocultas con costes que crecen con n, los tiempos aumentan más que lo supuesto por el análisis teórico del algoritmo implementado. Sin embargo, la complejidad teórica la mantendré como O(n) ya que a la hora de analizar puramente la complejidad del algoritmo per se, seria esa.

EJERCICIO 2

n	T(ms)
8	7
16	16
32	36

Jaime González Díaz
UO306054

64	88
100	137
128	168
256	366
512	909
1024	1790
2048	3190
4096	6443
8192	13021
16384	27621
32768	59641
65536	131550

Al igual que la implementación en java, la complejidad del algoritmo en Python es de O(n) por el mismo motivo que anteriormente, sin embargo Python tarda mas en ejecutar cada grafo debido a cualidades propias del lenguaje (es interpretado, es decir lee línea a línea el código mientras lo ejecuta, lo que tarda más que una compilación como la de java; el tipado dinámico…), además de que java lo compila con un compilador JIT, lo que acelera sus tiempos.

Práctica P5

Definición del estado:

Los índices de la tabla dp[i][j] se definen como:

- i: número de vehículos considerados en el problema. -
  j: metros ocupados en babor.

Si dp[i][j] es true, significa que se pueden colocar los primeros i vehículos ocupando j metros en babor.

No es necesario almacenar el valor de estribor ya que este se puede calcular. Como los vehículos se cargan siempre en el mismo orden, uno detrás del anterior, la suma de i vehículos siempre será conocida. Por lo que, en vez de almacenar la longitud de estribor, podemos calcularlo haciendo la resta de sumaLongVehiculos[i] – j, dándonos dicho valor sin tener que almacenarlo.

Relación de recurrencia:

La formulación matemática del estado base del algoritmo que permite calcular el estado (i,j) a partir de los estados anteriores es la siguiente:

<u>Caso base:</u>

dp[0][0]=true dp[0][j]=

false , ∀j>0

Es decir, cuando el número de vehículos es 0, el número de metros a babor ocupados tiene que ser siempre 0.

Recurrencia:

Para i = 1, …,N y j = 0, …,L:

dp[i][j] = true $\Leftrightarrow$ (dp[i-1][j] = true $\wedge \sum_{k=1}^{i} vk - j \leq L$) $\vee$ (dp[i−1][j−vi]=true$\wedge$j−vi ≥0)

Condición de parada: $\nexists$ j∈[0,L]:dp[i][j]=true

Complejidad:

La complejidad del algoritmo se puede definir como O(N*L), donde N es el número de vehículos que usamos en el problema y L es la longitud máxima permitida de cada carril. Esto debido a que el algoritmo recorre iterativamente N elementos y, dentro de cada iteración de N, recorre iterativamente L elementos. Dentro de estas iteraciones, los cálculos son de complejidad O(1). Por lo que, calculando la complejidad, podemos llegar a la conclusión de que el resultado es N*L*1 = O(N*L).

Implementación:

La implementación del algoritmo para resolver el problema se encuentra en la clase Ferry.java dentro del proyecto Java en el directorio de la entrega.
Dicha implementación supera todos los test propuestos con un acierto del 100%.

Práctica P6

TIEMPOS DEL ALGORITMO DE LOS CONTENEDORES:

Estas pruebas se hicieron en x=800 repeticiones para que los tiempos sean más altos y así sea más visible

Test	T(ms)
Test00	79
Test01	74
Test02	52
Test03	42
Test04	95
Test05	73
Test06	63
Test07	53
Test08	10991
Test09	FdT

Este problema normalmente representaría una subida de tiempos siempre que se aumente el tamaño del problema, pero al usar backtracking e implementar poda para optimizarlo todavía más, la subida de tiempos solo se nota cuando el problema tiene una

cantidad de objetos mucho mayor (test08, donde el tiempo es más de 100 veces mayor, o test09 donde el tiempo se aleja muchísimo de los anteriores) ya que, aunque la poda siga funcionando, el número de nodos prometedores crece de forma exponencial.
Si no se hubiese aplicado poda la complejidad del algoritmo seria O(m^n), siendo m el número de contenedores y n el número de objetos.
Sin embargo, con poda aplicada (siendo la versión actual y final del código) no hay una complejidad exacta de notación Big O que pueda representarla, por lo que técnicamente podríamos decir que su complejidad depende de la eficacia de la función de poda al aplicarse al problema y de cómo estén ordenados los datos, sí que podemos asegurar que es más eficiente que su versión sin poda ya que la función de poda permite "adelgazar" el árbol al descartar las soluciones que no vayan a ser las mejores. El peor caso sigue siendo exponencial como lo es al no aplicar poda.

Práctica P6 EXTRA

TABLA DE TIEMPOS ALGORITMO DE LAS N REINAS

El algoritmo se medirá en distintos tamaños del problema con la ejecución -XInt de Java para poder ver mejor la curva factorial de los tiempos del problema:

e	Tiempo (ms)
1	0
2	0
3	0
4	0
5	0
6	0
7	1
8	2
9	7
10	29
11	129
12	672
13	3914
14	23572
15	152793

La complejidad temporal del algoritmo es de O(n!), ya que es una implementación que resuelve el problema de las N reinas mediante backtracking pero que en cada nivel del árbol de llamadas el tamaño del nivel se reduce en 1, ya que primero analiza 4 columnas, luego una vez seleccionada la primera columna analiza 3, luego 2, luego 1… eso en el caso de un tablero de 4x4 (n=4). Por lo que la complejidad del tiempo es factorial, cuya explosión de tiempo es a partir de n=12, donde cada tiempo posterior es más de 5 veces mayor que el anterior, llegando así a un tiempo altísimo, dando un error de memoria por

el Heap de java en n=16 (por lo menos al ejecutarlo sin optimizaciones). Además, la complejidad espacial del algoritmo es de O(n).

Práctica P7

Heurístico aplicado

El heurístico aplicado consiste en sumar los pesos de todos los objetos que quedan por guardar y dividir ese valor entre la capacidad de los contenedores (todos tienen la misma capacidad). Este resultado, redondeado hacia arriba, es el número de contenedores mínimo de contenedores necesarios para meter todos los objetos en algún contenedor. Si la suma de los contenedores que ya están siendo usados más el mínimo de contenedores necesarios es igual o superior al mejor número actual, se descarta esta opción, ya que ya no será la solución óptima.
Este heurístico es mucho mejor que el anterior de cara a la búsqueda del mejor resultado, ya que ahorra mucho tiempo y omite todas las ramas del árbol de decisiones que se sabe de antemano que no van a ser la mejor respuesta, resolviendo así en segundos problemas que con el anterior heurístico podría llevar horas.

Tabla de tiempos comparativa

Para comparar los tiempos entre el anterior heurístico y el actual, usaremos un número de 800 repeticiones:

Test	T(ms) anterior	T(ms) nuevo
Test00	79	80
Test01	74	70
Test02	52	48
Test03	42	58
Test04	95	62
Test05	73	69
Test06	63	82
Test07	53	85
Test08	10991	108
Test09	FdT	77

En los test del 0 al 7, los tiempos mantienen un resultado relativamente similar entre ellos, ya que, aunque el nuevo algoritmo tenga un mejor heurístico, el problema no es tan complejo como para que el anterior (que ya implementaba poda) no lo pueda resolver también en poco tiempo. La diferencia real se ve en los últimos dos test, donde el test08 se resuelve 11 veces más rápido con el nuevo heurístico, y el test09 directamente se resuelve en 77ms con el nuevo heurístico mientras que con el anterior duraba una cantidad de tiempo no asumible.
El heurístico mejora significativamente la resolución de problemas altamente complejos o que le podrían llevar al anterior algoritmo una exploración de ramas absurdamente grande, error que evitamos con el nuevo al ignorar todos esos recorridos de ramas innecesarios. Sin embargo, en problemas más simples, tras varias ejecuciones, los tiempos de ambos algoritmos suelen ser similares con un margen de error de unos 5 a 10ms, con algún pico de diferencia de 30ms.

Jaime González Díaz
UO306054